

Sujet de Lab : Algorithmes d'Ensemble

Durée : 2h30

Objectif pédagogique : Maîtriser les algorithmes d'ensemble (Bagging, Boosting, Stacking, Voting) pour lutter contre le surapprentissage et améliorer les performances des modèles prédictifs.

Livraison : Un dépôt **GitHub** (repository) contenant votre code (scripts Python ou Notebooks), ainsi qu'un fichier **README.md** incluant le tableau de synthèse rempli. Le lien du dépôt devra être partagé à l'enseignant en fin de séance, **idéalement pour 17h**. Les rendus envoyés après cette heure permettront à l'équipe pédagogique d'identifier les étudiants ayant rencontré des difficultés majeures et nécessitant un accompagnement. Regroupez l'ensemble des liens dans un fichier unique (doc ou sheet) pour toute la cohorte, puis envoyez-le à henitsoa.rasoanandrianina@ingedata.ai.

Contexte

Dans ce lab, vous allez explorer différentes méthodes pour combiner des apprenants faibles (*weak learners*) afin de créer un modèle robuste et performant. Le but n'est pas seulement d'utiliser les bibliothèques existantes, mais de comprendre la mécanique interne de la réduction de variance (Bagging) et de la réduction de biais (Boosting). *Ce serait trop facile de vous aider d'un LLM pour faire le lab, mais pour votre développement personnel, essayez de coder par vous-même !*

Partie 0 : Configuration et Données (10 min)

0.1. Packages requis

Assurez-vous d'avoir installé les bibliothèques Python suivantes dans votre environnement (via **pip install** ou **conda install**) :

- **numpy**
- **pandas**
- **matplotlib**
- **scikit-learn**
- **xgboost**

- `scipy`

0.2. Jeu de données

Nous utiliserons un jeu de données synthétique généré via la fonction `make_classification` (issue de la librairie **Scikit-Learn** / `sklearn`) pour nous concentrer sur la modélisation plutôt que sur le nettoyage de données. Le jeu de données contient 2500 échantillons, 20 features (dont 12 informatives) et représente un problème de classification binaire.

Partie 1 : Bagging & Random Forest (40 min)

1.1. Modèle de base (Baseline)

Entraînez un arbre de décision (`DecisionTreeClassifier`) sans limite de profondeur. Évaluez son score (Accuracy) sur l'ensemble d'entraînement et de test. Que remarquez-vous vis-à-vis du surapprentissage ?

1.2. Implémentation manuelle du Bagging

Implémentez un algorithme de Bagging "à la main" :

- Créez une boucle générant $B = 50$ échantillons bootstrap (tirage avec remise).
- Entraînez un arbre sur chaque échantillon.
- Réalisez une prédiction par vote majoritaire sur l'ensemble de test.

1.3. Random Forest

Utilisez l'implémentation `RandomForestClassifier` de Scikit-Learn. Faites varier le paramètre `max_features` et visualisez l'importance des variables.

Question de réflexion : Pourquoi la limitation des variables à chaque nœud (`max_features`) améliore-t-elle la diversité des arbres par rapport à un Bagging classique ?

Partie 2 : Boosting (50 min)

2.1. AdaBoost

Le Boosting construit des modèles séquentiellement, chaque modèle corrigeant les erreurs du précédent. Entraînez un `AdaBoostClassifier` composé de "stumps" (arbres de profondeur 1). Analysez l'impact du `learning_rate`.

2.2. Gradient Boosting & XGBoost

Utilisez la librairie `xgboost` (XGBClassifier). Comparez ses performances et son temps d'exécution par rapport aux méthodes précédentes. Utilisez le paramètre `eval_set` pour tracer la courbe d'apprentissage (Log-Loss en fonction du nombre d'itérations) et identifiez le point où le modèle commence à surapprendre.

Partie 3 : Métamodèles — Voting & Stacking (35 min)

Combinez des modèles hétérogènes (par exemple : Régression Logistique, SVM, Random Forest, XGBoost).

- **Voting Classifieur** : Comparez le Hard Voting (vote sur la classe majoritaire) et le Soft Voting (moyenne des probabilités de chaque modèle).
- **Stacking Classifieur** : Entraînez un méta-modèle (ex: Régression Logistique) qui apprendra à combiner optimalement les prédictions des modèles de base pour fournir la décision finale.

Partie 4 : Bilan des performances (15 min)

Complétez le tableau récapitulatif dans le fichier `README.md` de votre dépôt GitHub avec les métriques suivantes pour chaque modèle que vous avez implémenté.

Modèle	Accuracy (Test)	Temps d'entraînement (s)	Observations / Surapprentissage
Arbre Seul			
Bagging manuel			
Random Forest			
AdaBoost			
XGBoost			
Stacking			

Bon courage ! N'hésitez pas à consulter la documentation officielle de Scikit-Learn et XGBoost durant la séance.